After Deep CFR: four modern poker AIs

Pluribus, ReBeL, Player of Games, AlphaHoldem — a pedagogical tour

Companion volume to *Deep CFR, explained*

PokerTrainer documentation

June 2026

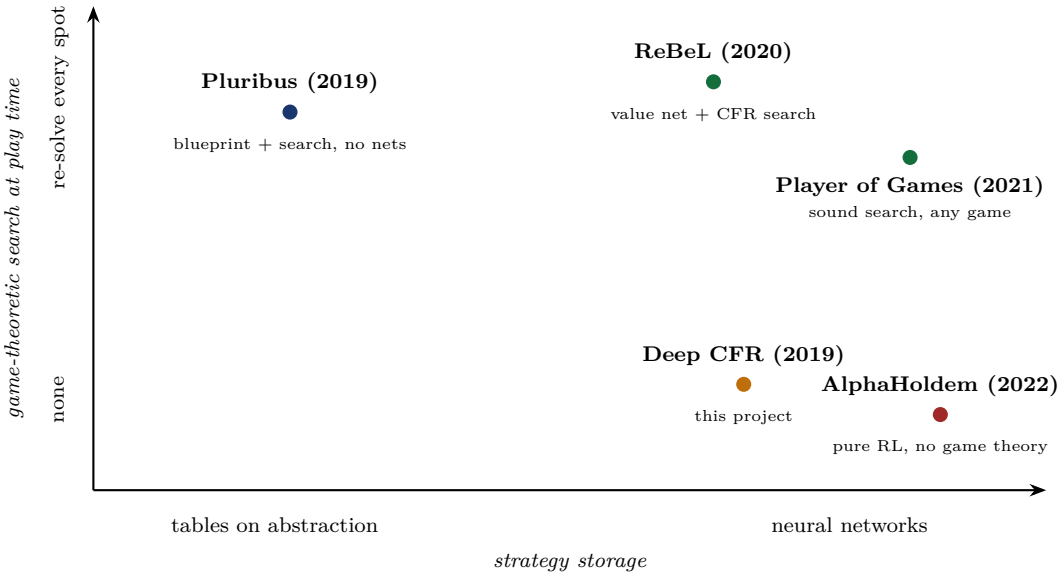
Abstract

The companion tutorial *Deep CFR, explained* covered the theory line from regret matching to Deep CFR. This volume takes the next step and dissects four landmark systems that followed: **Pluribus** (superhuman six-player poker with *no* neural network), **ReBeL** (reinforcement learning + search on *belief states*), **Player of Games** (one sound algorithm for chess, Go *and* poker), and **AlphaHoldem** (end-to-end deep RL, no search at all). For each: the one core idea, how it works mechanically (diagrams + Python pseudo-code), what it achieved, and what it costs. A final section compares all of them — including Deep CFR — on one table and draws lessons for this project.

Contents

1 The design space: one map before four deep dives

Every poker AI answers two independent questions. **(1) Where does game-theoretic reasoning happen?** Offline only (train a strategy, then just play it), or also *online* (re-solve the current situation at the table)? **(2) What stores the strategy?** Tables over abstracted infosets, or neural networks? The four systems in this volume occupy four distinct corners:



Intuition

Read the map as a tug-of-war between two resources: **offline training compute** (bottom row pays it all upfront, then plays instantly) and **online thinking time** (top row keeps solving at the table, which buys precision but costs seconds and engineering). The diagonal from Pluribus to AlphaHoldem is also a diagonal of *decreasing game theory and increasing deep learning.*

1.1 Refresher: the CFR core that three of the four systems share

Pluribus fills its blueprint with it, ReBeL runs it inside every subgame, Player of Games runs its regret-update phase with it — only AlphaHoldem drops it. So here is tabular CFR once, compactly, with its two classic upgrades shown inline (the full pedagogical build-up lives in the companion volume, Sections 2–3):

Listing 1: CFR in one block. `reach[p]` is the probability that player p's own choices lead here; regrets are weighted by the *opponent's* reach, the average strategy by *your own*. The commented lines show the vanilla / CFR$^+$ variants of the update; the active lines are Linear CFR (weight everything by t), which is what Pluribus and Deep CFR use.

```
R, S = tables_of_zeros(), tables_of_zeros()  # regrets / average strategy

def cfr(state, t, reach):
    if state.is_terminal():
        return state.payoffs()                    # (u_0, u_1), zero-sum
    I, p  = infoset(state), state.to_act()
    sigma = regret_matching(R[I])                 # positive part, normalized
    v, v_I = {}, [0.0, 0.0]
    for a in state.legal():
        r = list(reach)
        r[p] *= sigma[a]                          # my move scales MY reach
```

```
        v[a] = cfr(state.child(a), t, r)
        v_I  = [x + sigma[a] * y for x, y in zip(v_I, v[a])]
    for a in state.legal():
        # vanilla CFR:  R[I][a] +=     reach[1-p] * (v[a][p] - v_I[p])
        # CFR+:         R[I][a]  = max(R[I][a] + reach[1-p] * (..), 0)
        R[I][a] += t * reach[1 - p] * (v[a][p] - v_I[p])   # Linear CFR
    S[I] += t * reach[p] * sigma          # the average strategy = the output
    return v_I

def solve(T):
    for t in range(1, T + 1):
        cfr(deal_random_hand(), t, reach=[1.0, 1.0])   # chance sampled
    return normalize(S)                   # converges to Nash (2 players)
```

2 Pluribus: superhuman six-player poker, no neural network

Paper: Brown & Sandholm, *Superhuman AI for multiplayer poker*, Science 2019. **One idea:** a coarse offline "blueprint" strategy plus cheap *depth-limited* online re-solving is enough — even with six players, even with tables instead of networks.

2.1 Why six players changes the rules of the game

Everything in the Deep CFR tutorial relied on the two-player zero-sum correspondence: regret minimization ⇒ Nash, and Nash ⇒ unbeatable. With six players *both* halves break:

> In games with 3+ players, computing a Nash equilibrium is computationally intractable, and — worse — *playing* one is no longer a guarantee: if the others deviate, your equilibrium share can lose. Pluribus therefore drops the theoretical target entirely and aims for an *empirical* one: a strategy that consistently beats elite humans. CFR-style self-play is kept as a *heuristic* that happens to produce strong, balanced play.

2.2 Stage 1 (offline): the blueprint

The blueprint is classic pre-deep-learning machinery, exactly what Deep CFR was designed to replace — and Pluribus shows it is still viable when paired with search:

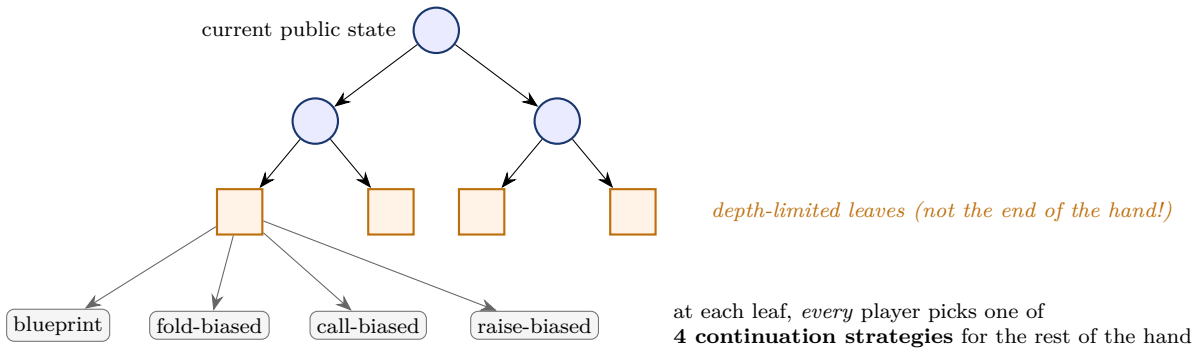
- **Information abstraction:** strategically similar hands are bucketed (k-means on equity distributions), shrinking 10^{161} situations to a tractable table.
- **Action abstraction:** only a handful of bet sizes exist in the abstract game.
- **Linear MCCFR** (the same external-sampling traversals as our trainer, plus the same linear t-weighting) fills the regret tables — this is exactly the Linear-CFR update of Listing 1, run with sampling.

The famous headline: the blueprint trained in **8 days on a 64-core server**, under 512 GB RAM, **no GPU** — roughly $150 of cloud compute, versus millions of dollars of TPU time for Go-class systems.

2.3 Stage 2 (online): depth-limited re-solving with continuation strategies

The blueprint alone is too coarse to beat pros (the abstraction blurs exact bet sizes and hand distinctions). At the table, Pluribus re-solves the *current subgame* finely. The problem: a subgame

in no-limit poker can still reach the end of the hand — far too deep to solve in seconds. The fix is the paper's key invention:



Intuition

Why continuation strategies make shallow leaves honest. If you evaluate a leaf with a *single* fixed policy (say, the blueprint), the search can "cheat": it steers play toward leaves where the blueprint plays badly for the opponent, producing a strategy that only works if the opponent cooperates. Letting every player *choose among several continuations* at the leaf is a miniature game: the value of the leaf is what you can get when the opponent picks their best continuation *against you*. The search can no longer assume a docile opponent — at a tiny fraction of the cost of solving to the end of the hand.

Listing 2: Pluribus, schematically. The blueprint is Deep CFR's ancestor (same traversals, tables instead of nets); play time adds depth-limited re-solving.

```
def blueprint(T):                          # OFFLINE, once, ~8 days on CPUs
    for t in range(1, T + 1):
        for p in range(6):                 # six players!
            mccfr_traverse(deal(), p, t)   # external sampling, linear weights
    return regret_tables                   # indexed by ABSTRACT infosets

def act(public_state, blueprint):          # ONLINE, every decision
    if early_and_on_tree(public_state):
        return sample(blueprint[abstract(public_state)])
    G = subgame(public_state, depth_limit=few_actions)
    for leaf in G.leaves():
        # each player picks 1 of 4 continuations at the leaf -> the leaf
        # itself becomes a small game (no fixed-policy cheating)
        leaf.attach_choices([blueprint, fold_bias, call_bias, raise_bias])
    sigma = linear_cfr(G)                  # re-solve the enriched subgame
    return sample(sigma[my_infoset])
```

Results. Against elite professionals (including World Series champions), in both 5 humans + 1 AI and 1 human + 5 copies formats, Pluribus won decisively. Each decision takes seconds on two CPUs.

Watch out

Pluribus is often summarized as "deep learning beats poker pros". It contains **no neural network at all**. Its lesson is the opposite of the hype: *search and good abstractions are devastatingly cost-effective*. When you read a new-method paper, always ask what a tuned blueprint-plus-search baseline would have done.

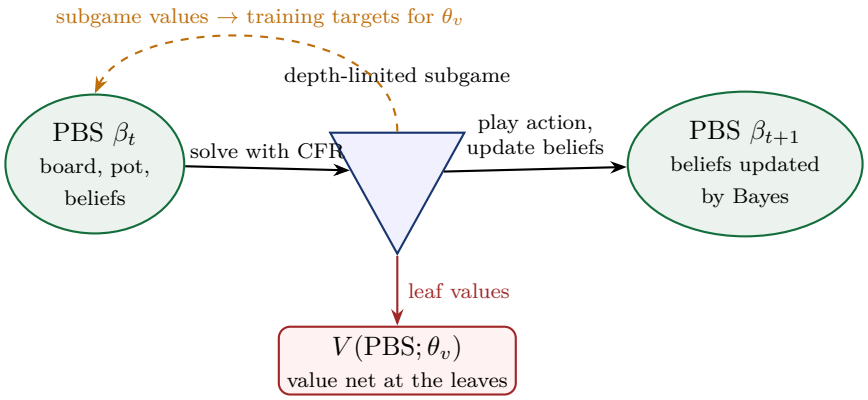
3 ReBeL: reinforcement learning + search on belief states

Paper: Brown, Bakhtin, Lerer, Gong, *Combining Deep Reinforcement Learning and Search for Imperfect-Information Games*, NeurIPS 2020. **One idea:** an imperfect-information game becomes a *perfect-information* game if the "state" is taken to be the **public belief state** — then AlphaZero-style self-play + search applies, with CFR doing the searching.

3.1 The conversion trick

Public belief state (PBS). Everything publicly visible (board, pot, betting history) *plus*, for each player, the probability distribution over their possible private hands, as computable by an outside observer who knows both players' strategies. In hold'em each belief is a vector over the 1326 possible hole-card pairs.

Why this works: if both players' policies are common knowledge, then Bayes' rule updates the beliefs after every public action — so the PBS evolves *deterministically given the policies*, like a board position. Hidden information has been traded for a bigger, continuous state. Values become well-defined functions of the PBS, so a neural network $V(\text{PBS})$ can be trained the way AlphaZero trains its value net.



3.2 The self-play loop

Listing 3: ReBeL's training loop (simplified). The same machinery runs at test time, minus the buffer writes.

```
def rebel_episode(theta_v, theta_pi, buffer):
    beta = initial_pbs()                        # blinds posted, uniform beliefs
    while not beta.is_terminal():
        G = depth_limited_subgame(beta)         # leaves valued by V(.; theta_v)
        policies = run_cfr(G, T_cfr,            # CFR-D: tabular CFR inside G,
                           warm_start=theta_pi)  # net leaf values
        # KEY SUBTLETY: play the policy of a RANDOM CFR iteration, not the
        # final average. This makes the value targets consistent with the
        # distribution of beliefs the next subgame will actually see.
        sigma = policies[randint(1, T_cfr)]
        buffer.add(beta, value_of_root(G, average(policies)))   # V target
        a = sample(sigma[my_private_infoset])
        beta = bayes_update(beta, sigma, a)  # PUBLIC update of both beliefs
    refit(theta_v, buffer)                   # like AlphaZero's value training
```

Intuition

Where Deep CFR and ReBeL put the neural network. Deep CFR's nets *replace the regret tables* inside one global solve. ReBeL's net replaces *the bottom of the search tree*: CFR itself stays tabular but only ever sees a small window of the game, and the net summarizes everything below the window. That is exactly AlphaZero's division of labor, transplanted to imperfect information — which is why the paper's title says "RL + Search".

Results. Superhuman heads-up no-limit hold'em with far less poker knowledge than DeepStack-/Libratus: it beat top HUNL professional Dong Kim by **165 mbb/hand over 7 500 hands** (Kim was the best-performing human of the Libratus match). Also solid play in Liar's Dice, showing generality.

Watch out

The PBS is the whole trick — and the whole bill. The belief vector needs one probability per private state: 1326 in hold'em is fine; a game with 10^{10} private states is not. And beliefs are only well-defined if the policy they are computed from is *common knowledge* — ReBeL effectively "announces" its strategy, which is safe at equilibrium but subtle to reason about away from it. Porting ReBeL to a new game starts with the question: *can I even write down the belief vector?*

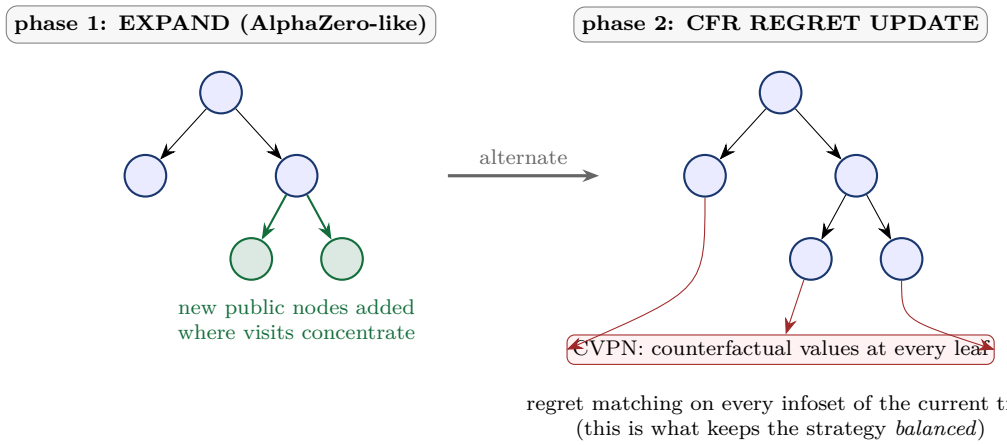
4 Player of Games: one sound algorithm for chess, Go and poker

Paper: Schmid et al., *Player of Games*, 2021; published in Science Advances 2023 under the name *Student of Games*. **One idea:** merge AlphaZero's *growing* search tree with CFR's *sound* handling of hidden information, so the same agent plays both kinds of games.

4.1 What "sound" buys you

AlphaZero's MCTS is built for perfect information; applied naively to poker it converges to exploitable strategies (it has no concept of balancing a range). ReBeL is sound but poker-shaped. Player of Games (PoG) asks: what is the *minimal* unification? Its answer has two components:

- a **counterfactual value-and-policy network (CVPN)**: given a public state plus beliefs (like ReBeL's PBS), it outputs *per-infoset counterfactual values* for both players and a policy prior. In a perfect-information game the beliefs are trivial and it degrades exactly to AlphaZero's value+policy net.
- **growing-tree CFR (GT-CFR)**: the search keeps a small *public* tree and alternates two phases — *expand* the tree where the policy wants to visit (AlphaZero-style), then run *CFR regret updates* over the whole current tree with CVPN values at the leaves.



Listing 4: GT-CFR and sound self-play (schematic).

```
def gt_cfr(beliefs, cvpn, n_rounds):
    tree = PublicTree(root=beliefs)              # starts tiny
    for _ in range(n_rounds):
        for _ in range(E):                       #  EXPAND
            leaf = descend_by_policy_and_visits(tree)
            tree.add_children(leaf, prior=cvpn.policy(leaf))
        for _ in range(R):                       #  REGRET UPDATE
            cfr_iteration(tree, leaf_values=cvpn.values)
    return tree.root_policy(), tree.root_values()

def sound_self_play(cvpn):
    while True:
        traj = play_game(act=lambda b: gt_cfr(b, cvpn, N))
        # every position where the net was queried at a leaf goes into a
        # QUERY BUFFER; a background process re-solves those positions
        # (recursively, with more search) to produce improved value targets
        train(cvpn, value_targets_from(query_buffer), policy_targets_from(traj))
```

Intuition

The query buffer is the conceptual heart of the training. The net is trained *at exactly the places where the search trusted it* — each leaf query becomes a promise to go back later, search deeper from that spot, and return with a better value to learn from. Compare our project's stage-1 oracle: the principle "validate the function where you actually rely on it" is the same; PoG just turns it into the training signal itself.

Results. One set of weights per game, same algorithm: expert-level chess and Go (below a specialized AlphaZero at equal compute — the price of generality), beats Slumbot (the strongest open heads-up poker bot) and is not exploited by a local best-response probe, and defeats the state of the art in Scotland Yard, a hide-and-seek board game with imperfect information.

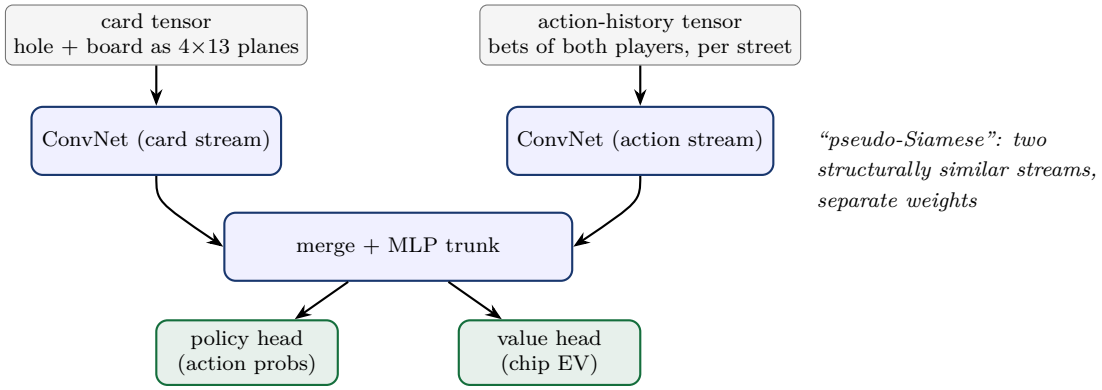
Watch out

Soundness here is a statement about the *search*, given good values: more search cannot make play more exploitable. It is not a global optimality proof, and in perfect-information games generality costs head-to-head strength versus a specialist. "One algorithm for everything" and "the best algorithm for each thing" remain different targets.

5 AlphaHoldem: what if we drop the game theory entirely?

Paper: Zhao, Yan, Li, Li, Xing, *AlphaHoldem: High-Performance Artificial Intelligence for Heads-Up No-Limit Poker via End-to-End Reinforcement Learning*, AAAI 2022. **One idea:** no CFR, no search, no beliefs — a single network maps the observation directly to an action, trained with a carefully stabilized PPO against a pool of past selves.

5.1 Architecture: two eyes, one brain



Note what is *absent*: no belief vector, no infoset enumeration, no regret anything. The action history tensor plays the role our token sequence plays in this project — it is the only channel through which the network can infer what the opponent's range looks like.

5.2 Making PPO survive poker

Vanilla PPO collapses on poker: rewards span ±200 big blinds with brutal variance, and importance ratios explode on rare lines. AlphaHoldem's **Trinal-Clip PPO** adds two clamps to the standard clipped loss:

Listing 5: Trinal-Clip PPO (the three clips). delta1 caps the policy ratio when the advantage is negative; delta2/delta3 cap the value target at what is physically winnable/losable in the hand.

```
def trinal_clip_losses(ratio, adv, v_pred, v_target, eps, d1, d2, d3):
    # 1+2) standard PPO clip, PLUS an upper cap d1 (> 1+eps) on the ratio
    #      when adv < 0 -- a huge ratio times a negative advantage otherwise
    #      yields an unbounded, variance-exploding policy loss
    r = clip(ratio, 1 - eps, 1 + eps) if adv >= 0 else \
        clip(ratio, 1 - eps, d1)
    policy_loss = -min(ratio * adv, r * adv)
    # 3) value target clipped to the chips actually reachable this hand
    #    (d2 = max losable, d3 = max winnable -- pot/stack dependent)
    value_loss = (v_pred - clip(v_target, -d2, d3)) ** 2
    return policy_loss, value_loss
```

5.3 K-best self-play against strategy cycling

Naive self-play (always train against the latest self) famously *cycles* in poker: agent B beats A, C beats B, and C loses to A again — rock-paper-scissors in strategy space, no monotone progress. AlphaHoldem trains each new agent against the K **best historical versions** simultaneously, scored by an Elo-like rating; a new agent enters the pool only if it earns its place. (AlphaStar's league is the same idea at larger scale.)

Listing 6: K-best self-play.

```
pool = [random_init()]
while training:
    learner = clone(best(pool))
    for _ in range(n_steps):
        opponent = sample(pool)                    # mix over the K best
        rollouts = play(learner, opponent)
        update(learner, trinal_clip_ppo(rollouts))
    pool = top_k(pool + [learner], K, by=elo)    # survival of the fittest
```

Results. Over 100 000 evaluation hands, AlphaHoldem beats Slumbot and a DeepStack re-implementation, after **3 days of training on one 8-GPU server**, and then decides in **2.9 ms** on a single GPU — roughly a thousand times faster than search-based systems.

Watch out

No theory, no certificate. Nothing bounds AlphaHoldem's exploitability; "beats Slumbot" does not imply "cannot be exploited by a strategy that hunts for its leaks" — and follow-up work has indeed found pure-RL poker agents exploitable. This is the precise trade this corner of the design-space map makes: spectacular speed and simplicity, no safety net. Our project's self-play collapses (the 1M-step incoherent policy) were a small taste of the same phenomenon — and our stage-1 Nash oracle is exactly the kind of certificate AlphaHoldem lacks.

6 Side by side, and what this project takes from each

	Deep CFR	Pluribus	ReBeL	PoG/SoG	AlphaHoldem
year	2019	2019	2020	2021	2022
core engine	CFR	CFR	CFR+RL	CFR+MCTS	PPO
neural nets	value+policy	**none**	value+policy	CVPN	policy+value
search at play time	no	yes	yes	yes	no
belief states needed	no	no	yes	yes	no
abstraction needed	no	yes	no	no	no
players	2	**6**	2	2	2
theoretical target	Nash	none (empirical)	Nash	sound search	none
training hardware	GPUs	64 CPU cores, 8 days	many GPUs	TPUs	8 GPUs, 3 days
decision latency	ms	seconds	seconds	seconds	**2.9 ms**

Intuition

Choosing for this project. PokerTrainer sits in the Deep CFR cell on purpose: no abstraction to hand-craft (the token transformer reads raw infosets), no play-time search infrastructure (one network ships in the Flutter inspector and the C ABI), and a real theoretical target (Nash) that gives us measurable progress — which the stage-1 push/fold oracle exploits directly. The map's other corners tell us what we are giving up: Pluribus-style re-solving would buy precision at the cost of an online solver; ReBeL/PoG belief machinery would buy soundness-at-depth at the cost of 1326-dim belief plumbing; AlphaHoldem's pure RL would buy simplicity at the cost of the very guarantees we built the oracle to enforce.

Concrete techniques worth borrowing here, in increasing order of effort:

1. **K-best opponent pools (AlphaHoldem)** — our evaluation matches already revealed self-play drift; rating checkpoints with Elo and evaluating new ones against the K best is cheap and directly transplantable.

2. **Reward/value clipping by reachable chips (AlphaHoldem)** — the same physical bounds (± effective stack) we used to reason about regret scale apply to any value-learning component we add.
3. **Continuation-strategy leaf evaluation (Pluribus)** — if we ever add depth limits beyond the current crude bootstrap (flagged in `cfr_coro.py`), the 4-continuation construction is the principled upgrade.
4. **Query-buffer-style targeted validation (PoG)** — extend the stage-1 oracle idea: log the infosets where the AdvNet's regrets most influenced traversals, and audit/validate exactly there.

7 Further reading

- Brown, Sandholm (2019). *Superhuman AI for multiplayer poker* (Pluribus). Science.
- Brown, Sandholm (2018). *Depth-Limited Solving for Imperfect-Information Games* — the continuation-strategies construction Pluribus builds on.
- Brown, Bakhtin, Lerer, Gong (2020). *Combining Deep Reinforcement Learning and Search for Imperfect-Information Games* (ReBeL). NeurIPS.
- Schmid et al. (2021/2023). *Player of Games / Student of Games: A unified learning algorithm for both perfect and imperfect information games.* Science Advances.
- Zhao, Yan, Li, Li, Xing (2022). *AlphaHoldem: High-Performance Artificial Intelligence for Heads-Up No-Limit Poker via End-to-End Reinforcement Learning.* AAAI.
- Moravčík et al. (2017). *DeepStack* — the origin of learned counterfactual values at depth limits, ancestor of both ReBeL and PoG.
- Companion volume: *Deep CFR, explained* (`docs/deep_cfr_tutorial/`) — regret matching, CFR, MCCFR, Deep CFR, and this project's curriculum.